

Ανακεφαλαίωση: βασικά προβλήματα του συστήματος αξιολόγησης:

1. Missing ή ατελείς βαθμολογίες → Normalized Weighted Scoring
2. Μη αξιόπιστες κριτικές → Robust & Bayesian Filtering
3. Ίδιο βάρος σε δύσκολες/εύκολες δουλειές → Difficulty Coefficients
4. Παλιές κριτικές μετράνε το ίδιο → Time Decay
5. Όλοι βαθμολογούνται 4.5–5.0 → Variance Stretching για ξεκάθαρη κατάταξη
6. Πλαστές/ύποπτες κριτικές → Anomaly Detection (4 statistical signatures)
7. Ασυνεπές scoring → Ενιαίος optimized μαθηματικός τύπος

1.Πρόβλημα: Λείπουν βαθμολογίες σε ορισμένα κριτήρια

Π.χ. ο πελάτης δεν έβαλε αστέρια σε “Communication” ή “Professionalism”.

Λύση: Normalized Weighted Score

$$S_i = \frac{\sum_{k \in K_i} w_k \cdot r_{ik}}{\sum_{k \in K_i} w_k}$$

Όπου K_i είναι μόνο οι παράμετροι που όντως βαθμολογήθηκαν.

Ο αλγόριθμος λαμβάνει υπόψη **μόνο** τα κριτήρια που έχουν βαθμολογηθεί, με αναπροσαρμογή των βαρών ώστε το score να είναι δίκαιο και σταθερό.

2.Πρόβλημα: Κριτικές αναξιόπιστες — υπερβολικά χαμηλές ή χαριστικές

Υπάρχουν περιπτώσεις:

- κακόβουλης χαμηλής βαθμολογίας (π.χ. 1 ★ χωρίς λόγο)
- υπερβολικά θετικές “ψεύτικες” βαθμολογίες (5 ★ χωρίς βάση)

Λύση: Robust Statistics

- Trimmed Mean (αφαίρεση ανώτερου/κατώτερου 10%)

$$S_{\text{robust}} = \text{TrimmedMean}_{10\%}(S_1, \dots, S_N)$$

- Bayesian smoothing ώστε μία κακή κριτική να μην καταστρέφει το profile

$$S_{\text{Bayes}} = \frac{\mu \cdot \alpha + \sum S_i}{\alpha + N}$$

$\mu=4.5$ (expected platform average) , $\alpha=10$ (10 jobs)

- Απομόνωση «εξαιρέσεων» χωρίς να πέφτει ο μέσος όρος απότομα (Η Να μην ανεβαίνει)

3.Πρόβλημα: Όλες οι δουλειές έχουν το ίδιο βάρος

Μια δύσκολη εργασία πρέπει να μετράει περισσότερο από μια απλή.

Λύση: Difficulty Weight

Κάθε εργασία λαμβάνει difficulty coefficient (1.0 έως 2.0).

Έτσι:

- δύσκολες δουλειές = μετρούν περισσότερο
- εύκολες δουλειές = λιγότερο
 - εύκολη $\rightarrow d_i = 1.0$
 - φυσιολογική $\rightarrow d_i = 1.3$
 - δύσκολη $\rightarrow d_i = 1.6$
 - πολύ δύσκολη $\rightarrow d_i = 2.0$

Τότε στο JSS μπαίνει:

$$S_i^* = d_i \cdot S_i$$

4.Πρόβλημα: Οι παλιές κριτικές μετράνε όσο οι καινούριες

Αποτέλεσμα: δεν φαίνεται η βελτίωση ενός τεχνικού.

Λύση: Time Decay

Εισάγεται συντελεστής απομείωσης:

$e^{-\lambda t}$

με $\lambda \approx 0.05$.

Οι πρόσφατες κριτικές έχουν μεγαλύτερη αξία.

$$\text{Decay}(t_i) = e^{-\lambda t_i}$$

όπου:

- t_i = ηλικία κριτικής σε μήνες
- $\lambda = 0.05$ (μετά από 12 μήνες το βάρος $\approx 55\%$)

5.Πρόβλημα: Score Inflation — όλοι παίρνουν 4.5–5.0

Άρα οι πολύ καλοί τεχνικοί δεν ξεχωρίζουν.

Λύση: Variance Stretching

Αυξάνεται η διαφορά μεταξύ 4.5, 4.7, 4.8, 5.0, ώστε να υπάρχει πραγματικός διαχωρισμός.

Με scaling factor (π.χ. 1.8×) η κατάταξη γίνεται πιο καθαρή.

$$S_{\text{stretched}} = 4.0 + (S - 4.0) \cdot 1.8$$

Δηλαδή:

6.Πρόβλημα: Πιθανές πλαστές ή ύποπτες κριτικές

- 4.5 → 4.9
- 4.7 → 5.1 (capped to 5.0)

- 4.3 → 4.54

Π.χ.:

- πολλές κριτικές την ίδια μέρα
- reviews με copy-paste μοτίβα
- rating που δεν ταιριάζει με το comment
- ξαφνικές απότομες μεταβολές (positive ή negative attacks)

Λύση: Anomaly Detection Module (Statistical Signatures)

Το σύστημα αναλύει κάθε κριτική με 4 υπο-αλγόριθμους:

A. Z-Score Outlier

Αν η βαθμολογία αποκλίνει υπερβολικά από το προφίλ του τεχνικού.

$$Z = \frac{r - \mu}{\sigma}$$

Αν $|Z| > 2.5 \rightarrow$ ύποπτο.

B. Temporal Spike Detection

Αν πολλαπλές κριτικές μπαίνουν ξαφνικά μαζί.

(1 αν υπάρχει spike την ίδια μέρα, αλλιώς 0)

$$T_i = spike(i)$$

C. Sentiment-Rating Mismatch

Αν το κείμενο είναι θετικό αλλά τα αστέρια χαμηλά (ή το αντίστροφο).

$$M_i = |\text{Sentiment}(\text{comment}_i) - S_i^{\text{core}}|$$

D. Duplicate Linguistic Patterns

Αν τα σχόλια μοιάζουν ύποπτα μεταξύ τους (εντοπισμός γλωσσικών μοτίβων):

- πανομοιότυπες προτάσεις
- ίδιος ρυθμός γραφής
- ίδια στίξη

(μάλλον απάτη).

$$A_i = w_1 Z_i + w_2 T_i + w_3 M_i + w_4 P_i$$

όπου:

- Z_i : z-score outlier
- T_i : temporal spike
- M_i : sentiment mismatch
- P_i : pattern similarity
- w_j : βάρη (ρυθμιζόμενα)

Αν $A_i > \text{threshold}$, η κριτική χαρακτηρίζεται ύποπτη.

Αποτέλεσμα:

Αν η κριτική είναι ύποπτη:

$$Penalty_i = 0.5$$

αλλιώς:

$$Penalty_i = 1$$

Το effective weight γίνεται:

$$W_i^{eff} = d_i \cdot e^{-\lambda t_i} \cdot Penalty_i$$

Οι ύποπτες κριτικές μπαίνουν σε **reduced weight mode** (50% βάρος).

7.Πρόβλημα: Ο χρόνος, η δυσκολία και οι ανωμαλίες δεν συνδυάζονται σε έναν ενιαίο τύπο

Χωρίς ενιαίο μοντέλο, η βαθμολογία παραμένει ασυνεπής.

Λύση: Ενιαίος ολοκληρωμένος τύπος JSS

Ο τελικός τύπος ενσωματώνει:

- normalized ratings
- weighted criteria
- time decay
- difficulty
- robust statistics
- bayesian correction
- anomaly penalties
- variance stretching

Έτσι το output είναι **δίκαιο, σταθερό, προστατευμένο από απάτη και αντικειμενικό.**

$$\text{JSS} = 4.0 + \left(\frac{\mu\alpha + \sum_{i \in T} \left(\frac{\sum_{k \in K_i} w_k r_{ik}}{\sum_{k \in K_i} w_k} \right) \cdot d_i \cdot e^{-\lambda t_i} \cdot \text{Penalty}_i}{\alpha + |T|} - 4.0 \right) \cdot \gamma$$

όπου:

- $\text{Penalty}_i = 1$ ή 0.5 αν $A_i > \theta$
- T είναι το trimmed (10%) σύνολο
- γ είναι το stretching factor
- α και μ το Bayesian prior
- d_i difficulty
- $e^{-\lambda t_i}$ time decay
- r_{ik}, w_k τα ratings & weights

Τελικό αποτέλεσμα:

Ένα ενιαίο, δίκαιο, ανθεκτικό και μαθηματικά βελτιστοποιημένο JSS που προστατεύει την πλατφόρμα και τους τεχνικούς.

Στο τέλος του project θα δοθούν:

- Ο τελικός μαθηματικός τύπος (καθαρός, απλός, εξηγημένος).
- Τεχνική τεκμηρίωση.
- **Προαιρετικά:** code implementation

Εκτιμώμενος χρόνος ολοκλήρωσης: **2–3 ημέρες**, ανάλογα με την απαίτηση τεκμηρίωσης και αν χρειάζεται υλοποίηση σε κώδικα.

Mathematical Optimization offer:

180€ χωρίς υλοποίηση σε κώδικα

250€ με κώδικα

Θα χρειαστεί να δείτε τα παρακάτω για να μου πείτε την προσωπική σας γνώμη:

Συνιστώμενες προεπιλεγμένες ρυθμίσεις (πρακτικές τιμές)

1. **bayesMu** = 4.5

(Η «βασική αναμενόμενη μέση βαθμολογία» της πλατφόρμας. Προστατεύει νέους τεχνικούς με λίγες κριτικές ώστε να μη φαίνονται κακοί ή υπερβολικά καλοί από 1–2 κριτικές.)

2. **bayesAlpha** = 10 (virtual reviews)

Πόσο «ισχυρή» είναι η προκαθορισμένη βάση (σαν να έχει κάθε τεχνικός 10 εικονικές κριτικές). Για να μην ανεβοκατεβαίνει το score απότομα με λίγες κριτικές.

3. **decayLambda** = 0.05 (σε μήνες — μετά από 12 μήνες βάρος $\approx 55\%$)

Με αυτή την τιμή, μετά από ~ 1 χρόνο μια κριτική έχει περίπου τη μισή βαρύτητα.

4. **trimPercent** = 0.10 (αφαίρεση κορυφών 10% top & bottom)

Αφαιρούμε το χειρότερο 10% και το καλύτερο 10% των κριτικών. Αν πολλά fake reviews, ανέβασέ το (0.15).

5. **stretchFactor** γ = 1.8

Ανοίγει την κλίμακα των βαθμών ώστε να ξεχωρίζουν καλύτερα οι καλοί από τους μέτριους. Αν η πλατφόρμα θέλει μεγαλύτερη διαφοροποίηση, πήγαινε το στο 2.

6. **anomalyThreshold** = 1.5...2.5 (εξαρτάται από scaling, ξεκίνα με 2.0)

Το όριο που ορίζει πότε μια κριτική θεωρείται ύποπτη (Ευαισθησία του anomaly module). Αν πολλά fake reviews τότε χαμήλωσε το πχ 1.5.

7. **penaltySuspicious** = 0.5

Αν μια κριτική θεωρηθεί ύποπτη, μετράει με μισό βάρος. Αν θέλεις πιο αυστηρή προστασία βάλε 0.3 (μειώνει ακόμη περισσότερο)

17/11/2025

Μπούτου Θεοδοσία

theodosia2001mp@gmail.com